42.5 部署上线

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~370k tokens，输出 ~5k tokens（本节）

MiniClaw 的功能开发完成后，本节将指导读者将其部署到生产环境。我们将覆盖 Docker 容器化、守护进程配置和安全加固三个方面。

42.5.1 Docker 容器化

Dockerfile

参考 OpenClaw 的多阶段构建（第 29 章），为 MiniClaw 编写 Dockerfile：

```dockerfile
# Dockerfile

# 阶段 1：构建
FROM node:22-slim AS builder
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN corepack enable && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build

# 阶段 2：运行
FROM node:22-slim AS runtime
RUN groupadd -r miniclaw && useradd -r -g miniclaw -m miniclaw
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json .
COPY --from=builder /app/ui ./ui

# 非 root 用户运行
USER miniclaw

# 数据目录
VOLUME /home/miniclaw/.miniclaw
ENV NODE_ENV=production

EXPOSE 3000 8080
CMD ["node", "dist/index.js"]
```

关键安全措施：

措施	说明
多阶段构建	生产镜像不包含构建工具和源码
非 root 用户	使用 `miniclaw` 用户运行，降低权限
`--frozen-lockfile`	确保依赖版本与开发环境一致
VOLUME 挂载	数据持久化在宿主机上

docker-compose.yml

```yaml
# docker-compose.yml

version: "3.8"

services:
  miniclaw:
    build: .
    ports:
      - "3000:3000"
      - "8080:8080"
    volumes:
      - miniclaw-data:/home/miniclaw/.miniclaw
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - TELEGRAM_BOT_TOKEN=${TELEGRAM_BOT_TOKEN}
    restart: unless-stopped
    # 资源限制
    deploy:
      resources:
        limits:
          memory: 512M
          cpus: "1.0"

volumes:
  miniclaw-data:
```

启动命令

```
# 构建镜像
docker compose build

# 创建 .env 文件
echo "ANTHROPIC_API_KEY=sk-ant-..." > .env
echo "TELEGRAM_BOT_TOKEN=123456:ABC..." >> .env

# 启动
docker compose up -d

# 查看日志
docker compose logs -f
```

42.5.2 守护进程配置

对于不使用 Docker 的部署方式，需要将 MiniClaw 配置为系统守护进程。

systemd（Linux）

```
# /etc/systemd/system/miniclaw.service

[Unit]
Description=MiniClaw AI Assistant
After=network.target

[Service]
Type=simple
User=miniclaw
WorkingDirectory=/opt/miniclaw
ExecStart=/usr/bin/node dist/index.js
Restart=on-failure
RestartSec=5s

# 环境变量
EnvironmentFile=/opt/miniclaw/.env

# 安全加固
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/opt/miniclaw/.miniclaw

# 资源限制
MemoryMax=512M
CPUQuota=100%

[Install]
WantedBy=multi-user.target


# 安装服务
sudo systemctl daemon-reload
sudo systemctl enable miniclaw
sudo systemctl start miniclaw

# 查看状态
sudo systemctl status miniclaw
sudo journalctl -u miniclaw -f
```

launchd（macOS）

```xml
<!-- ~/Library/LaunchAgents/com.miniclaw.gateway.plist -->

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>
  <string>com.miniclaw.gateway</string>
  <key>ProgramArguments</key>
  <array>
    <string>/usr/local/bin/node</string>
    <string>/opt/miniclaw/dist/index.js</string>
  </array>
  <key>WorkingDirectory</key>
  <string>/opt/miniclaw</string>
  <key>RunAtLoad</key>
  <true/>
  <key>KeepAlive</key>
  <true/>
  <key>StandardOutPath</key>
  <string>/tmp/miniclaw-stdout.log</string>
  <key>StandardErrorPath</key>
  <string>/tmp/miniclaw-stderr.log</string>
  <key>EnvironmentVariables</key>
  <dict>
    <key>NODE_ENV</key>
    <string>production</string>
  </dict>
</dict>
</plist>


# 加载服务
launchctl load ~/Library/LaunchAgents/com.miniclaw.gateway.plist

# 停止服务
launchctl unload ~/Library/LaunchAgents/com.miniclaw.gateway.plist
```

42.5.3 安全加固

参考 OpenClaw 的安全模型（第 24 章），MiniClaw 的最小安全清单：

1. 认证保护

```
// 在 Gateway 中添加简单的 Token 认证

const GATEWAY_TOKEN = process.env.MINICLAW_TOKEN;

gateway.addMiddleware((client, frame) => {
  if (!GATEWAY_TOKEN) return true;  // 未配置则跳过
  if (client.authenticated) return true;

  if (frame.type === "req" && frame.method === "auth") {
    const token = (frame.params as { token: string })?.token;
    if (token === GATEWAY_TOKEN) {
      client.authenticated = true;
      return true;
    }
    return false;  // 认证失败
  }
  return false;  // 未认证的请求拒绝
});
```

2. 工具安全

```
// Bash 工具增强安全

const BLOCKED_COMMANDS = [
  /rm\s+-rf\s+\//,
  /mkfs/,
  /dd\s+if=\/dev/,
  />(\/etc|\/var|\/usr)/,
  /curl.*\|\s*(bash|sh)/,
  /wget.*\|\s*(bash|sh)/,
];

const ALLOWED_PATHS = [
  process.cwd(),
  path.join(os.homedir(), ".miniclaw"),
];

function isSafeCommand(command: string): boolean {
  for (const pattern of BLOCKED_COMMANDS) {
    if (pattern.test(command)) return false;
  }
  return true;
}

function isSafePath(filePath: string): boolean {
  const resolved = path.resolve(filePath);
  return ALLOWED_PATHS.some(allowed => resolved.startsWith(allowed));
}
```

3. 输入消毒

```
// 防止提示词注入

function sanitizeInput(message: string): string {
  // 移除可能的系统提示注入尝试
  const dangerous = [
    /ignore (previous|above|all) instructions/i,
    /you are now/i,
    /system:\s/i,
    /\[SYSTEM\]/i,
  ];

  let sanitized = message;
  for (const pattern of dangerous) {
    if (pattern.test(sanitized)) {
      sanitized = `[User message - potentially contains instruction override attempt]\n
      break;
    }
  }
  return sanitized;
}
```

4. 反向代理

在生产环境中，建议使用 Caddy 或 Nginx 作为反向代理：

```
# Caddyfile

miniclaw.example.com {
    # WebSocket 代理
    @ws {
        header Connection *Upgrade*
        header Upgrade websocket
    }
    reverse_proxy @ws localhost:3000

    # WebChat UI
    reverse_proxy /chat/* localhost:8080

    # 自动 HTTPS
    tls {
        protocols tls1.3
    }

    # 速率限制
    rate_limit {
        zone dynamic {
            key {remote_host}
            events 60
            window 1m
        }
    }
}
```

安全检查清单

项目	状态	说明
Gateway Token 认证	必需	防止未授权访问
HTTPS / WSS	必需	加密传输
非 root 运行	必需	最小权限
工具命令白名单	建议	限制危险操作
文件路径限制	建议	限制在工作目录内
输入消毒	建议	防止提示词注入

项目	状态	说明
速率限制	建议	防止滥用
日志审计	可选	记录所有工具调用
自动备份	可选	定期备份会话和记忆数据

本节小结

1. **Docker 容器化** 使用多阶段构建，以非 root 用户运行，数据通过 volume 持久化。
2. **守护进程** 支持 systemd（Linux）和 launchd（macOS），配置自动重启和资源限制。
3. **安全加固** 覆盖四个层面：Gateway 认证、工具安全、输入消毒、反向代理。
4. **反向代理** 推荐 Caddy（自动 HTTPS），配置 WebSocket 代理和速率限制。
5. 部署后应逐项检查安全清单，确保生产环境的安全性。